
Advanced Data Mining

Sequential Pattern Mining

Outline

- Introduction
 - Definitions
 - Sequential Pattern Mining Algorithms
-

Introduction

- ❑ More and more data!
- ❑ A need to analyze data to find **interesting patterns**
- ❑ **Pattern mining**: using algorithms to find interesting patterns in data.
- ❑ An important type of data is **sequences**.
- ❑ Today, we will discuss how to analyze sequences to find **sequential rules**.

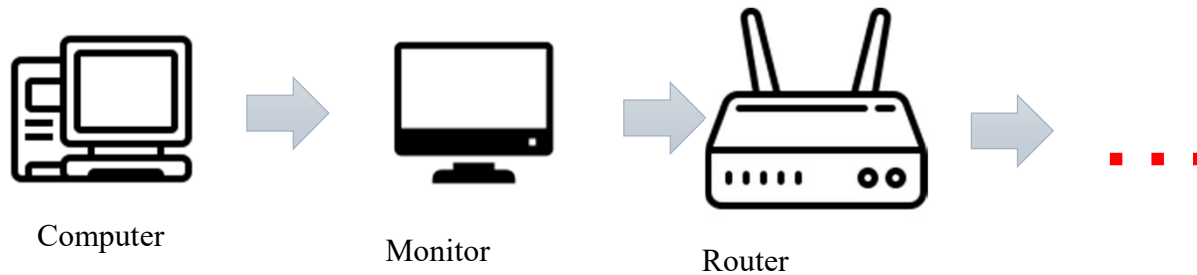
Why sequential pattern mining?

- ☐ Association rule mining does not consider the order of transactions.
 - ☐ Time or order in which actions appear or happen can be relevant in decision making
 - ☐ Many applications of sequential pattern mining
 - Customer shopping patterns, telephone calling patterns
 - ☐ E.g., first buy computer, then CD-ROMS, software, within 3 mos.
 - Natural disasters (e.g., earthquake, hurricane)
 - Disease and treatment
 - Stock market fluctuation
 - Weblog click stream analysis
 - DNA sequence analysis
-

What is a sequence?

Sequence: an ordered list of symbols

Sequence of purchases



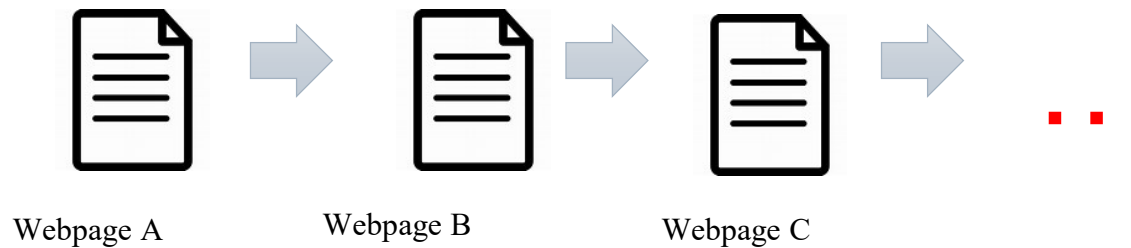
Sequence of words

Where → **are** → **you** → **going?**

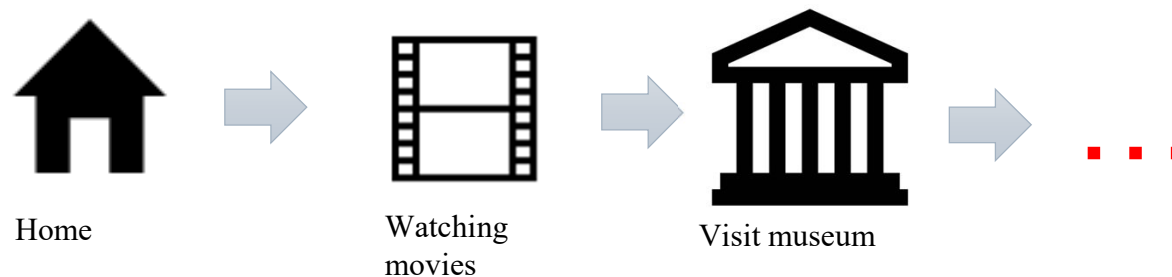
What is a discrete sequence?

Sequence: an ordered list of symbols

Sequences of webpage clicks



Sequences of activities



Definition: Itemset

Let there be a **set of items** (symbols) called I .

Example: $I = \{a, b, c, d, e, f, g\}$

An itemset is a set of **items** that is a subset of I .

Example: $\{a, b, c\}$ is an itemset containing 3 items

$\{d, e\}$ is an itemset containing 2 items

- Note: an itemset cannot contain a same item twice.
 - An itemset having k items is called a *k-itemset*.
-

Formal Definition of a Sequence

- A sequence is an ordered list of elements (transactions)

$$S = \langle e_1 e_2 e_3 \dots \rangle$$

- Each element contains a collection of events (items)

$$e_i = \{i_1, i_2, \dots, i_k\}$$

- Example 1: $\langle \{a, b\}, \{c\} \rangle$ is a sequence containing two elements (transactions, itemsets).

It means that a customer purchased a and b at the same time and then purchased c .

- Length of a sequence, $|s|$, is given by the number of elements of the sequence
 - A k -sequence is a sequence that contains k events (items)
-

Sequence Databases

- A sequence database consists of one or more sequences
 - A sequence consists of ordered elements or events
 - Each element can be a set of items or a single item (a singleton set)
 - Each sequence has a unique identifier
- Example 1: This is a sequence database with four sequences $SD=\{10, 20, 30, 40\}$:

Sequence database

SID	sequence
10	<a(abc)(ac)d(cf)>
20	<(ad)c(bc)(ae)>
30	<(ef)(ab)(df)cb>
40	<eg(af)cbc>

Sequence Databases

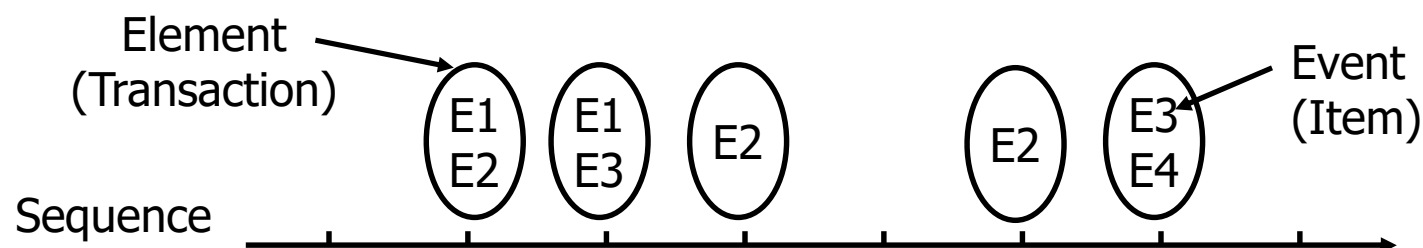
- ❑ Converts the original transaction database into a database of customer sequences
- ❑ Example:

SID	sequence
10	<a(abc)(ac)d(cf)>
20	<(ad)c(bc)(ae)>
30	<(ef)(ab)(df)cb>
40	<eg(af)cbc>

Customer ID	Items bought	Date
10	a	10
10	a b c	15
10	a c	20
10	d	25
10	c f	30
20	a d	15
20	c	20
20	b c	25
20	a e	30
30	e f	20
	...	
40	e g	10
	...	

Sequence Databases

Sequence Database	Sequence	Element (Transaction)	Event (Item)
Customer	Purchase history of a given customer	A set of items bought by a customer at time t	Books, diary products, CDs, etc
Web Data	Browsing activity of a particular Web visitor	A collection of files viewed by a Web visitor after a single mouse click	Home page, index page, contact info, etc
Event data	History of events generated by a given sensor	Events triggered by a sensor at time t	Types of alarms generated by sensors
Genome sequences	DNA sequence of a particular species	An element of the DNA sequence	Bases A,T,G,C



Sequence Databases

A sequence database

SID	sequence
10	<a(abc)(ac)d(cf)>
20	<(ad)c(bc)(ae)>
30	<(ef)(ab)(df)cb>
40	<eg(af)cbc>

<a(abc)(ac)d(cf)> - 5 elements, 9 items

<a(abc)(ac)d(cf)> - 9-sequence

Order doesn't
Matter (list)

<a(abc)(ac)d(cf)> = <a(cba)(ac)d(cf)>
<a(abc)(ac)d(cf)> ≠ <a(ac)(abc)d(cf)>

Order matters (sequence)

Subsequence vs. super sequence

- A sequence is an ordered list of events, denoted $\langle e_1 e_2 \dots e_l \rangle$
- Given two sequences $\alpha = \langle a_1 a_2 \dots a_n \rangle$ and $\beta = \langle b_1 b_2 \dots b_m \rangle$
- α is called a subsequence of β , denoted as $\alpha \subseteq \beta$, if there exist integers $1 \leq j_1 < j_2 < \dots < j_n \leq m$ such that $a_1 \subseteq b_{j_1}, a_2 \subseteq b_{j_2}, \dots, a_n \subseteq b_{j_n}$
- β is a super sequence of α – E.g. $\alpha = \langle (\text{ab}), \text{d} \rangle$ and $\beta = \langle (\text{abc}), (\text{de}) \rangle$
- Examples:
 - $\langle (\text{ab}), \text{d} \rangle$ is a subsequence of $\langle (\text{abc}), (\text{de}) \rangle$
 - $\langle 3, (4, 5), 8 \rangle$ is contained in (or is a subsequence of)
 $\langle 6, (3, 7), 9, (4, 5, 8), (3, 8) \rangle$
 - $\langle \text{a.html}, \text{c.html}, \text{f.html} \rangle \subseteq$
 $\langle \text{a.html}, \text{b.html}, \text{c.html}, \text{d.html}, \text{e.html}, \text{f.html}, \text{g.html} \rangle$

Formal Definition of a Subsequence

Data sequence	Subsequence	Contain?
$\langle \{2,4\} \{3,5,6\} \{8\} \rangle$	$\langle \{2\} \{8\} \rangle$	Yes
$\langle \{1,2\} \{3,4\} \rangle$	$\langle \{1\} \{2\} \rangle$	No
$\langle \{2,4\} \{2,4\} \{2,5\} \rangle$	$\langle \{2\} \{4\} \rangle$	Yes
$\langle \{2,4\} \{2,5\} \{4,5\} \rangle$	$\langle \{2\} \{4\} \{5\} \rangle$	No
$\langle \{2,4\} \{2,5\} \{4,5\} \rangle$	$\langle \{2\} \{5\} \{5\} \rangle$	Yes
$\langle \{2,4\} \{2,5\} \{4,5\} \rangle$	$\langle \{2, 4, 5\} \rangle$	No

Definition: Support of a sequence

The number of sequences in a **sequence database** SD that contain a sequence S_A is called the support of S_A . It is defined as:

$$\text{sup}(S_A) = |\{S \mid S \in D \text{ and } S_A \sqsubseteq S\}|$$

Example 1:

Sequence database

$S_1 =$	$\langle \{\textcolor{red}{a}, b\}, \{c\}, \{a\} \rangle$
$S_2 =$	$\langle \{\textcolor{red}{a}\}, \{b\}, \{c\} \rangle$
$S_3 =$	$\langle \{b\}, \{c\}, \{d\} \rangle$
$S_4 =$	$\langle \{b\}, \{\textcolor{red}{a}, b\}, \{c\} \rangle$

$$\text{sup}(\langle \{\textcolor{red}{a}\} \rangle) = 3$$

Definition: Support of a sequence

The number of sequences in a **sequence database** D that contain a sequence S_A is called the support of S_A . It is defined as:

$$\text{sup}(S_A) = |\{S \mid S \in D \text{ and } S_A \sqsubseteq S\}|$$

Example 2:

Sequence database

$S_1 =$	$\langle \{a, \textcolor{red}{b}\}, \{c\}, \{a\} \rangle$
$S_2 =$	$\langle \{a\}, \{\textcolor{red}{b}\}, \{c\} \rangle$
$S_3 =$	$\langle \{\textcolor{red}{b}\}, \{c\}, \{d\} \rangle$
$S_4 =$	$\langle \{\textcolor{red}{b}\}, \{a, \textcolor{red}{b}\}, \{c\} \rangle$

$$\text{sup}(\langle \{\textcolor{red}{b}\} \rangle) = 4$$

Definition: Support of a sequence

The number of sequences in a **sequence database** D that contain a sequence S_A is called the support of S_A . It is defined as:

$$\text{sup}(S_A) = |\{S \mid S \in D \text{ and } S_A \sqsubseteq S\}|$$

Example 3:

Sequence database

$S_1 =$	$\langle \{a, b\}, \{c\}, \{a\} \rangle$
$S_2 =$	$\langle \{a\}, \{b\}, \{c\} \rangle$
$S_3 =$	$\langle \{b\}, \{c\}, \{d\} \rangle$
$S_4 =$	$\langle \{b\}, \{a, b\}, \{c\} \rangle$

$$\text{sup}(\langle \{a\}, \{b\} \rangle) = 1$$

Definition: Support of a sequence

The number of sequences in a **sequence database** D that contain a sequence S_A is called the support of S_A . It is defined as:

$$\text{sup}(S_A) = |\{S \mid S \in D \text{ and } S_A \sqsubseteq S\}|$$

Example 4: **Sequence database**

$S_1 =$	$\langle \{a, b\}, \{c\}, \{a\} \rangle$
$S_2 =$	$\langle \{a\}, \{b\}, \{c\} \rangle$
$S_3 =$	$\langle \{b\}, \{c\}, \{d\} \rangle$
$S_4 =$	$\langle \{b\}, \{a, b\}, \{c\} \rangle$

$$\text{sup}(\langle \{a, b\} \rangle) = 2$$

What Is Sequential Pattern Mining?

- ❑ It is a popular data mining task, introduced in 1994 by Agrawal & Srikant.
 - ❑ The goal is to find all subsequences that appear frequently in a set of sequences.
 - ❑ **For example:**
 - find sequences of items purchased by many customers over time,
 - find sequences of locations frequently visited by tourists in a city,
 - Find sequences of words that appear frequently in a text.
-

What Is Sequential Pattern Mining?

- ❑ **Input:** A sequence database D and a minimum support threshold $minsup$
 - ❑ **Output:** All sequential patterns.
A sequential pattern is a subsequence S where $sup(S) \geq minsup$.
 - ❑ It is similar to the frequent itemsets mining, but with consideration of ordering.
 - ❑ The support of a subsequence s is defined as the fraction of data sequences that contain s
 - ❑ Sequential Pattern Mining is useful in many application, e.g. weblog analysis, financial market prediction, BioInformatics, etc.
-

What Is Sequential Pattern Mining?

- Given a set of sequences, find the complete set of *frequent* subsequences

A sequence database

SID	sequence
10	<a(<u>ab</u> c)(a <u>c</u>)d(cf)>
20	<(ad)c(bc)(ae)>
30	<(ef)(<u>ab</u>)(df) <u>c</u> b>
40	<eg(af)cbc>

Given support threshold $min_sup = 2$ (50%)
<(ab)c> is a sequential pattern
Its **support** is 50% (it appears in 50% of the sequences).

What Is Sequential Pattern Mining?

□ Given a set of sequences: A sequence database

SID	sequence
10	<a(<u>abc</u>)(a <u>c</u>)d(cf)>
20	<(ad)c(bc)(ae)>
30	<(ef)(<u>ab</u>)(df) <u>cb</u> >
40	<eg(af)cbc>

Ex: Given support threshold $min_sup = 2$

- $s1 = \langle c(ab) \rangle \rightarrow \text{supp}(s1) = ?$
- $s2 = \langle ca \rangle \rightarrow \text{supp}(s2) = ?$
- $s3 = \langle abc \rangle \rightarrow \text{supp}(s3) = ?$

$s1, s2, s3$ is a sequential pattern or not?

What Is Sequential Pattern Mining?

A sequence database

SID	sequence
10	<a(abc)(ac)d(cf)>
20	<(ad)c(bc)(ae)>
30	<(ef)(ab)(df)cb>
40	<eg(af)cbc>

$min_sup = 2$ (50%)

Solution – 53 frequent subsequences:

<a><aa> <ab> <a(bc)> <a(bc)a> <aba> <abc>

<(ab)> <(ab)c> <(ab)d> <(ab)f> <(ab)dc>

<ac>

<aca> <acb> <acc> <ad> <adc> <af>

 <ba> <bc> <(bc)> <(bc)a> <bd> <bdc>

<bf>

<c> <ca> <cb> <cc>

<d> <db> <dc> <dcb>

<e> <ea> <eab> <eac> <each> <eb> <ebc>

<ec>

<ecb> <ef> <efb> <efc> <efcb>

<f> <fb> <fbc> <fc> <fcb>

Extracting Sequential Patterns

- Given n events: $i_1, i_2, i_3, \dots, i_n$
- Candidate 1-subsequences: $\langle \{i_1\} \rangle, \langle \{i_2\} \rangle, \langle \{i_3\} \rangle, \dots, \langle \{i_n\} \rangle$
- Candidate 2-subsequences:
 $\langle \{i_1, i_2\} \rangle, \langle \{i_1, i_3\} \rangle, \dots,$
 $\langle \{i_1\} \{i_1\} \rangle, \langle \{i_1\} \{i_2\} \rangle, \dots, \langle \{i_n\} \{i_n\} \rangle$
- Candidate 3-subsequences:
 $\langle \{i_1, i_2, i_3\} \rangle, \langle \{i_1, i_2, i_4\} \rangle, \dots,$
 $\langle \{i_1, i_2\} \{i_1\} \rangle, \langle \{i_1, i_2\} \{i_2\} \rangle, \dots,$
 $\langle \{i_1\} \{i_1, i_2\} \rangle, \langle \{i_1\} \{i_1, i_3\} \rangle, \dots,$
 $\langle \{i_1\} \{i_1\} \{i_1\} \rangle, \langle \{i_1\} \{i_1\} \{i_2\} \rangle, \dots$

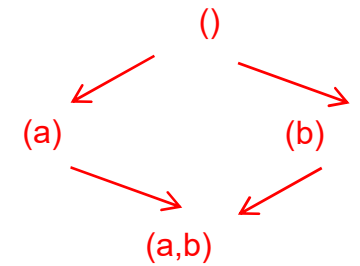
Extracting Sequential Patterns: Simple example

□ Given 2 events: a, b

□ Candidate 1-subsequences: $\langle \{a\} \rangle$, $\langle \{b\} \rangle$.

□ Candidate 2-subsequences:

$\langle \{a\} \{a\} \rangle$, $\langle \{a\} \{b\} \rangle$, $\langle \{b\} \{a\} \rangle$, $\langle \{b\} \{b\} \rangle$, $\langle \{a, b\} \rangle$.



Item-set patterns

□ Candidate 3-subsequences:

$\langle \{a\} \{a\} \{a\} \rangle$, $\langle \{a\} \{a\} \{b\} \rangle$, $\langle \{a\} \{b\} \{a\} \rangle$, $\langle \{a\} \{b\} \{b\} \rangle$,

$\langle \{b\} \{b\} \{b\} \rangle$, $\langle \{b\} \{b\} \{a\} \rangle$, $\langle \{b\} \{a\} \{b\} \rangle$, $\langle \{b\} \{a\} \{a\} \rangle$

$\langle \{a, b\} \{a\} \rangle$, $\langle \{a, b\} \{b\} \rangle$, $\langle \{a\} \{a, b\} \rangle$, $\langle \{b\} \{a, b\} \rangle$

Sequential Pattern Mining: Challenge

- Given a sequence: $\langle \{a\ b\} \ \{c\ d\ e\} \ \{f\} \ \{g\ h\ i\} \rangle$

■ Examples of subsequences:

 $\langle \{a\} \{c d\} \{f\} \{g\} \rangle, \langle \{c d e\} \rangle, \langle \{b\} \{g\} \rangle, \text{etc.}$

- How many k-subsequences can be extracted from a given n-sequence?

$$\langle \{a \ b\} \ \{c \ d \ e\} \ \{f\} \ \{g \ h \ i\} \rangle \quad n = 9$$

k=4:

Y _ _ Y Y _ _ Y

<{a} {d e} {i}>

Answer :

$$\binom{n}{k} = \binom{9}{4} = 126$$

Challenges on Sequential Pattern Mining

- ❑ A huge number of possible sequential patterns are hidden in databases
 - ❑ A mining algorithm should
 - find the complete set of patterns, when possible, satisfying the minimum support (frequency) threshold
 - be highly efficient, scalable, involving only a small number of database scans
 - be able to incorporate various kinds of user-specific constraints
-

Sequential Pattern Mining Algorithms

- ❑ **GSP**: R. Agrawal, and R. Srikant, Mining sequential patterns, ICDE 1995, pp. 3–14, 1995.
 - ❑ **SPAM**: Ayres, J. Flannick, J. Gehrke, and T. Yiu, Sequential pattern mining using a bitmap representation, KDD 2002, pp. 429–435, 2002.
 - ❑ **SPADE**: M. J. Zaki, SPADE: An efficient algorithm for mining frequent sequences, Machine learning, vol. 42(1-2), pp. 31–60, 2001.
 - ❑ **PrefixSpan**: J. Pei, et al. Mining sequential patterns by pattern-growth: The prefixspan approach, IEEE Transactions on knowledge and data engineering, vol. 16(11), pp. 1424–1440, 2004.
 - ❑ **CM-SPAM** and **CM-SPADE**: P. Fournier-Viger, A. Gomariz, M. Campos, and R. Thomas, Fast Vertical Mining of Sequential Patterns Using Co-occurrence Information, PAKDD 2014, pp. 40–52, 2014.
-

Sequential Pattern Mining Algorithms

- Apriori-based Approaches
 - GSP
 - SPADE
 - Pattern-Growth-based Approaches
 - FreeSpan
 - PrefixSpan
-

The Apriori Property of Sequential Patterns

- A basic property: Apriori (Agrawal & Srikant'94)
 - If a sequence S is not frequent, then none of the super-sequences of S is frequent
 - E.g, <hb> is infrequent → so do <hab> and <(ah)b>

Seq. ID	Sequence
10	<(bd)cb(ac)>
20	<(bf)(ce)b(fg)>
30	<(ah)(bf)abf>
40	<(be)(ce)d>
50	<a(bd)bcb(ade)>

Given support threshold $min_sup = 2$

Generalized Sequential Pattern (GSP) Algorithms

- **Step 1:**
 - Make the first pass over the sequence database D to yield all the 1-element frequent sequences
- **Step 2:**

Repeat until no new frequent sequences are found

 - **Candidate Generation:**
 - Merge pairs of frequent subsequences found in the $(k-1)th$ pass to generate candidate sequences that contain k items
 - **Candidate Pruning:**
 - Prune candidate k -sequences that contain infrequent $(k-1)$ -subsequences
 - **Support Counting:**
 - Make a new pass over the sequence database D to find the support for these candidate sequences
 - **Candidate Elimination:**
 - Eliminate candidate k -sequences whose actual support is less than *minsup*

Candidate Generation

□ Base case ($k=2$):

- Merging two frequent 1-sequences $\langle \{i_1\} \rangle$ and $\langle \{i_2\} \rangle$ will produce two candidate 2-sequences: $\langle \{i_1\} \{i_2\} \rangle$ and $\langle \{i_1 i_2\} \rangle$. (**Note:** $\langle \{i_1\} \rangle$ can be merged with itself to produce: $\langle \{i_1\} \{i_1\} \rangle$)

□ General case ($k>2$):

- A frequent $(k-1)$ -sequence w_1 is merged with another frequent $(k-1)$ -sequence w_2 to produce a candidate k -sequence if the subsequence obtained by removing the first event in w_1 is the same as the subsequence obtained by removing the last event in w_2
 - The resulting candidate after merging is given by the sequence w_1 extended with the last event of w_2 .
 - If the last two events in w_2 belong to the same element, then the last event in w_2 becomes part of the last element in w_1
 - Otherwise, the last event in w_2 becomes a separate element appended to the end of w_1

Candidate Generation Examples

- Merging the sequences

$w_1 = \langle \{1\} \{2\ 3\} \{4\} \rangle$ and $w_2 = \langle \{2\ 3\} \{4\ 5\} \rangle$

will produce the candidate sequence $\langle \{1\} \{2\ 3\} \{4\ 5\} \rangle$ because the last two events in w_2 (4 and 5) belong to the same element

- Merging the sequences

$w_1 = \langle \{1\} \{2\ 3\} \{4\} \rangle$ and $w_2 = \langle \{2\ 3\} \{4\} \{5\} \rangle$

will produce the candidate sequence $\langle \{1\} \{2\ 3\} \{4\} \{5\} \rangle$ because the last two events in w_2 (4 and 5) do not belong to the same element

- Merging $w_1 = \langle \{1\ 2\ 3\} \rangle$ and $w_2 = \langle \{2\ 3\ 4\} \rangle$

produces the candidate sequence $\langle \{1\ 2\ 3\ 4\} \rangle$ because the last element in w_2 has more than one event

- We do not have to merge the sequences

$w_1 = \langle \{1\} \{2\ 6\} \{4\} \rangle$ and $w_2 = \langle \{1\} \{2\} \{4\ 5\} \rangle$

to produce the candidate $\langle \{1\} \{2\ 6\} \{4\ 5\} \rangle$ because if the latter is a viable candidate, then it can be obtained by merging w_1 with

$\langle \{1\} \{2\ 6\} \{5\} \rangle$

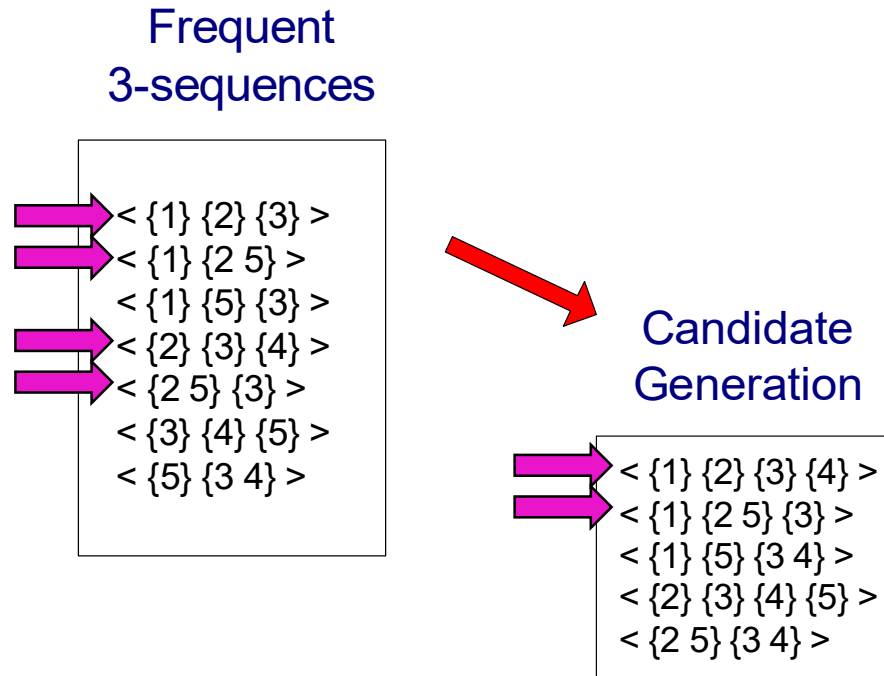
Candidate Generation: Examples (ctd)

- ☐ Can $\langle \{a\}, \{b\}, \{c\} \rangle$ merge with $\langle \{b\}, \{c\}, \{f\} \rangle$?
- ☐ Can $\langle \{a\}, \{b\}, \{c\} \rangle$ merge with $\langle \{b, c\}, \{f\} \rangle$?
- ☐ Can $\langle \{a\}, \{b\}, \{c\} \rangle$ merge with $\langle \{b\}, \{c, f\} \rangle$?
- ☐ Can $\langle \{a, b\}, \{c\} \rangle$ merge with $\langle \{b\}, \{c, f\} \rangle$?
- ☐ Can $\langle \{a, b, c\} \rangle$ merge with $\langle \{b, c, f\} \rangle$?
- ☐ Can $\langle \{a\} \rangle$ merge with $\langle \{a\} \rangle$?

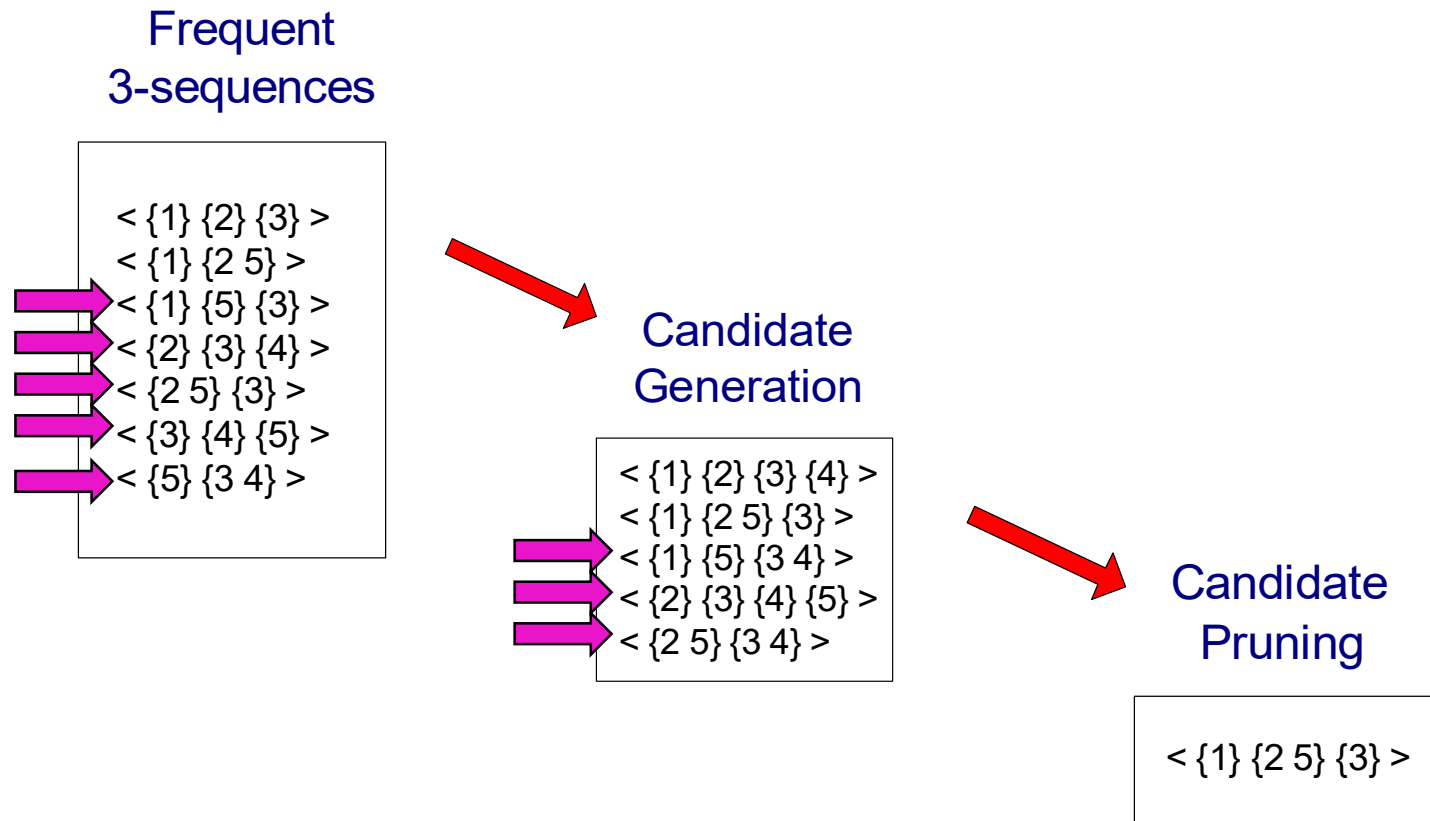
Candidate Generation: Examples (ctd)

- ❑ $\langle \{a\}, \{b\}, \{c\} \rangle$ can be merged with $\langle \{b\}, \{c\}, \{f\} \rangle$ to produce $\langle \{a\}, \{b\}, \{c\}, \{f\} \rangle$
- ❑ $\langle \{a\}, \{b\}, \{c\} \rangle$ cannot be merged with $\langle \{b, c\}, \{f\} \rangle$
- ❑ $\langle \{a\}, \{b\}, \{c\} \rangle$ can be merged with $\langle \{b\}, \{c, f\} \rangle$ to produce $\langle \{a\}, \{b\}, \{c, f\} \rangle$
- ❑ $\langle \{a, b\}, \{c\} \rangle$ can be merged with $\langle \{b\}, \{c, f\} \rangle$ to produce $\langle \{a, b\}, \{c, f\} \rangle$
- ❑ $\langle \{a, b, c\} \rangle$ can be merged with $\langle \{b, c, f\} \rangle$ to produce $\langle \{a, b, c, f\} \rangle$
- ❑ $\langle \{a\} \{b\} \{a\} \rangle$ can be merged with $\langle \{b\} \{a\} \{b\} \rangle$ to produce $\langle \{a\}, \{b\}, \{a\}, \{b\} \rangle$
- ❑ $\langle \{b\} \{a\} \{b\} \rangle$ can be merged with $\langle \{a\} \{b\} \{a\} \rangle$ to produce $\langle \{b\}, \{a\}, \{b\}, \{a\} \rangle$

Candidate Generation: Examples



Candidate Generation: Examples



Finding Length-1 Sequential Patterns

Seq. ID	Sequence
10	<(bd)cb(ac)>
20	<(bf)(ce)b(fg)>
30	<(ah)(bf)abf>
40	<(be)(ce)d>
50	<a(bd)bcb(ade)>

Given support threshold $min_sup = 2$

Cand	Sup
<a>	3
	5
<c>	4
<d>	3
<e>	3
<f>	2
<g>	1
<h>	1

- Initial candidates:
 <a>, , <c>, <d>, <e>, <f>, <g>, <h>
- Scan database once, count support for candidates

GSP: Generating Length-2 Candidates

51 length-2
Candidates

	<a>		<c>	<d>	<e>	<f>
<a>	<aa>	<ab>	<ac>	<ad>	<ae>	<af>
	<ba>	<bb>	<bc>	<bd>	<be>	<bf>
<c>	<ca>	<cb>	<cc>	<cd>	<ce>	<cf>
<d>	<da>	<db>	<dc>	<dd>	<de>	<df>
<e>	<ea>	<eb>	<ec>	<ed>	<ee>	<ef>
<f>	<fa>	<fb>	<fc>	<fd>	<fe>	<ff>

	<a>		<c>	<d>	<e>	<f>
<a>		<(ab)>	<(ac)>	<(ad)>	<(ae)>	<(af)>
			<(bc)>	<(bd)>	<(be)>	<(bf)>
<c>				<(cd)>	<(ce)>	<(cf)>
<d>					<(de)>	<(df)>
<e>						<(ef)>
<f>						

Without Apriori property,
 $8*8+8*7/2=92$ candidates

Apriori prunes
44.57% candidates

Finding Length-2 Sequential Patterns

- ❑ Scan database one more time, collect support count for each length-2 candidate
- ❑ There are 19 length-2 candidates which pass the minimum support threshold
 - They are length-2 sequential patterns

Finding Length-2 Sequential Patterns

19 Length-2
Sequential
Patterns

	<a>		<c>	<d>	<e>	<f>
<a>	<aa>:2	<ab>:2	<ac>:1	<ad>:1	<ae>:1	<af>:1
	<ba>:3	<bb>:4	<bc>:4	<bd>:2	<be>:3	<bf>:2
<c>	<ca>:2	<cb>:3	<cc>:1	<cd>:2	<ce>:1	<cf>:1
<d>	<da>:2	<db>:2	<dc>:2	<dd>:1	<de>:1	<df>:0
<e>	<ea>:0	<eb>:1	<ec>:1	<ed>:1	<ee>:1	<ef>:1
<f>	<fa>:1	<fb>:2	<fc>:1	<fd>:0	<fe>:1	<ff>

	<a>		<c>	<d>	<e>	<f>
<a>		<(ab)>:0	<(ac)>:1	<(ad)>:1	<(ae)>:1	<(af)>:0
			<(bc)>:0	<(bd)>:2	<(be)>:1	<(bf)>:2
<c>				<(cd)>:0	<(ce)>:2	<(cf)>:0
<d>					<(de)>:1	<(df)>:0
<e>						<(ef)>:0
<f>						

Generating Length-3 Candidates Patterns

- Generate Length-3 Candidates
 - Self-join length-2 sequential patterns
 - Based on the Apriori property
 - $\langle ab \rangle$, $\langle aa \rangle$ and $\langle ba \rangle$ are all length-2 sequential patterns $\rightarrow \langle aba \rangle$ is a length-3 candidate
 - 54 candidates are generated
 - $\langle (bd) \rangle$, $\langle bb \rangle$ and $\langle db \rangle$ are all length-2 sequential patterns $\rightarrow \langle (bd)b \rangle$ is a length-3 candidate
 - 27 candidates are generated
 - $a(bd)$, $(bd)a$, $b(bd)$, $(bd)b$, $(bd)c$, $(bd)d$, $(bd)e$, $(bd)f$, $c(bd)$, $d(bd)$, $f(bd)$, $a(bf)$, $(bf)a$, $(bf)b$, $b(bf)$, $(bf)c$, $(bf)d$, $(bf)e$, $(bf)f$, $c(bf)$, $d(bf)$, $f(bf)$, $b(ce)$, $(ce)a$, $(ce)b$, $(ce)d$, $d(ce)$

Finding Length-3 Sequential Patterns

- ❑ Scan database one more time, collect support count for each length-3 candidate
- ❑ There are 19 out of 81 candidates which pass the minimum support threshold
 - They are length-3 sequential patterns

The GSP Mining Process

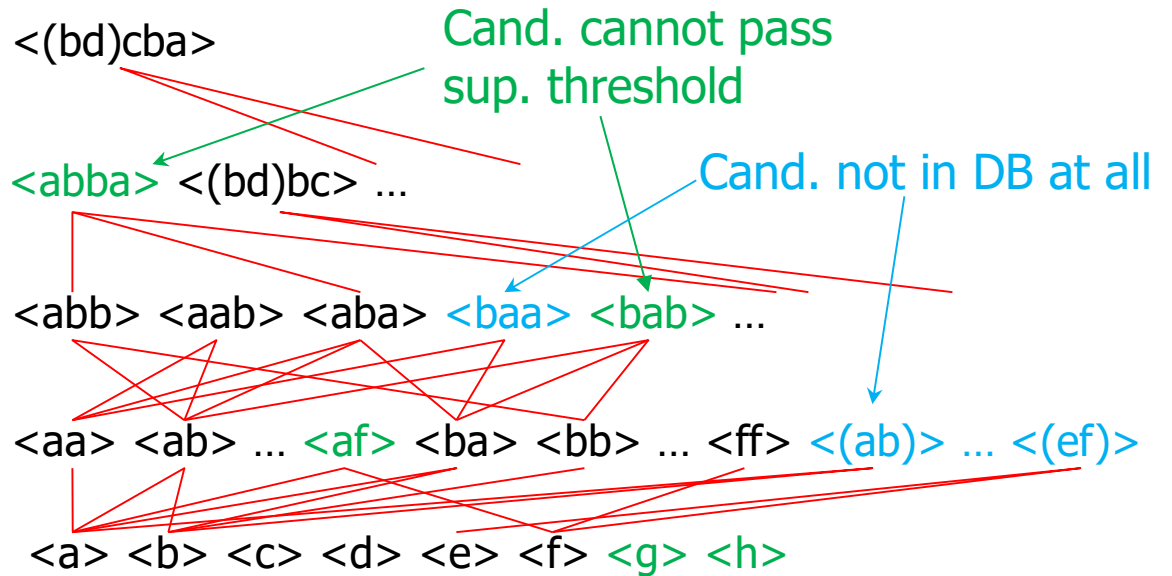
5th scan: 1 cand. 1 length-5 seq.
pat.

4th scan: 8 cand. 6 length-4 seq.
pat.

3rd scan: 46 cand. 19 length-3 seq.
pat. 20 cand. not in DB at all

2nd scan: 51 cand. 19 length-2 seq.
pat. 10 cand. not in DB at all

1st scan: 8 cand. 6 length-1 seq.
pat.



$min_sup = 2$

Seq. ID	Sequence
10	$\langle (bd)cb(ac) \rangle$
20	$\langle (bf)(ce)b(fg) \rangle$
30	$\langle (ah)(bf)abf \rangle$
40	$\langle (be)(ce)d \rangle$
50	$\langle a(bd)bcb(ade) \rangle$

Candidate Generate-and-test: Drawbacks

- A huge set of candidate sequences generated.
 - Especially 2-item candidate sequence.
 - Multiple Scans of database needed.
 - The length of each candidate grows by one at each database scan.
 - Inefficient for mining long sequential patterns.
 - A long pattern grow up from short patterns
 - The number of short patterns is exponential to the length of mined patterns.
-

Exercise 1

- Given sequence database and $min_sup = 80\%$. Find all sequential patterns.

Seq. ID	Sequence
10	<(bd)cb(ac)>
20	<(bf)(ce)b(fg)>
30	<(ah)(bf)abf>
40	<(be)(ce)d>
50	<a(bd)bcb(ade)>

Finding Length-1 Sequential Patterns

- Initial candidates:
 $\langle a \rangle, \langle b \rangle, \langle c \rangle, \langle d \rangle, \langle e \rangle, \langle f \rangle, \langle g \rangle, \langle h \rangle$
 - Scan database once, count support for candidates $\rightarrow F1 = \langle b \rangle, \langle c \rangle$
 - C2: $\langle bb \rangle, \langle bc \rangle, \langle cc \rangle, \langle cb \rangle, \langle (b,c) \rangle$
 - Scan database once, count support for candidates $\rightarrow \langle bb \rangle:4, \langle bc \rangle:4, \langle cc \rangle:1, \langle cb \rangle:2, \langle (b,c) \rangle:0 \rightarrow F2: \langle bb \rangle:4, \langle bc \rangle:4$
 - C3: $\langle bbc \rangle$
 - Scan database once, count support for candidates $\rightarrow \langle bbc \rangle:2$
- $\rightarrow F3 = \{\}$

Cand	Sup
$\langle a \rangle$	3
$\langle b \rangle$	5
$\langle c \rangle$	4
$\langle d \rangle$	3
$\langle e \rangle$	3
$\langle f \rangle$	2
$\langle g \rangle$	1
$\langle h \rangle$	1

Exercise 2

- Given sequence database and $min_sup = 80\%$. Find all sequential patterns.

SID	sequence
10	<a(abc)(ac)d(cf)>
20	<(ad)c(bc)(ae)>
30	<(ef)(ab)(df)cb>
40	<eg(af)cbc>

The SPADE Algorithm

- ❑ SPADE (Sequential PAttern Discovery using Equivalent Class)
developed by Zaki 2001
- ❑ A vertical format sequential pattern mining method
- ❑ A sequence database is mapped to a large set of Item: <SID, EID>
- ❑ Sequential pattern mining is performed by
 - growing the subsequences (patterns) one item at a time by Apriori candidate generation

The SPADE Algorithm (cont.)

SID	EID	Items
1	1	a
1	2	abc
1	3	ac
1	4	d
1	5	cf
2	1	ad
2	2	c
2	3	bc
2	4	ae
3	1	ef
3	2	ab
3	3	df
3	4	c
3	5	b
4	1	e
4	2	g
4	3	af
4	4	c
4	5	b
4	6	c

a		b		...
SID	EID	SID	EID	...
1	1	1	2	
1	2	2	3	
1	3	3	2	
2	1	3	5	
2	4	4	5	
3	2			
4	3			

ab			ba			...
SID	EID (a)	EID(b)	SID	EID (b)	EID(a)	...
1	1	2	1	2	3	
2	1	3	2	3	4	
3	2	5				
4	3	5				

aba				...
SID	EID (a)	EID(b)	EID(a)	...
1	1	2	3	
2	1	3	4	

Bottlenecks of GSP and SPADE

- A huge set of candidates could be generated

- 1,000 frequent length-1 sequences generate a huge number of length-2 candidates!

$$1000 \times 1000 + \frac{1000 \times 999}{2} = 1,499,500$$

- Multiple scans of database in mining

- Breadth-first search

- Mining long sequential patterns

- Needs an exponential number of short candidates

- A length-100 sequential pattern needs 10^{30} $\sum_{i=1}^{100} \binom{100}{i} = 2^{100} - 1 \approx 10^{30}$ candidate sequences!

PrefixSpan (Prefix-Projected Sequential Pattern Growth)

- ❑ PrefixSpan – Projection-based proposed by Jian Pei et al (2001)
 - But only prefix-based projection: less projections and quickly shrinking sequences
 - ❑ This algorithm is designed to only consider patterns that *exist* in the database.
 - ❑ This algorithm uses a concept of *database projection* and a *depth-first search*
 - ❑ This is not the most efficient algorithm, but it is simple and easy to extend, so it is popular.
-

Prefix of a Sequence

- Given two sequences $\alpha = \langle a_1 a_2 \dots a_n \rangle$ and $\beta = \langle b_1 b_2 \dots b_m \rangle$, $m \leq n$
- Sequence β is called a prefix of α if and only if:
 - $b_i = a_i$ for $i \leq m-1$;
 - $b_m \subseteq a_m$;
 - All the items in $(a_m - b_m)$ are alphabetically after those in b_m

Suffix of a Sequence

- Let $\alpha' = \langle a_1 a_2 \dots a_n \rangle$ be the projection of α w.r.t. prefix $\beta = \langle a_1 a_2 \dots a_{m-1} a'_m \rangle$ ($m \leq n$)
- Sequence $\gamma = \langle a''_m a_{m+1} \dots a_n \rangle$ is called the postfix of α w.r.t. prefix β , denoted as $\gamma = \alpha / \beta$, where $a''_m = (a_m - a'_m)$
- We also denote $\alpha = \beta \cdot \gamma$

Projection

- Given sequences α and β , such that β is a subsequence of α .
- A subsequence α' of sequence α is called a projection of α w.r.t. β prefix if and only if
 - α' has prefix β ;
 - There exist no proper super-sequence α'' of α' such that α'' is a subsequence of α and also has prefix β

Prefix and Suffix (Projection)

- $\langle a \rangle$, $\langle aa \rangle$, $\langle a(ab) \rangle$ and $\langle a(abc) \rangle$ are prefixes of sequence $\langle a(abc)(ac)d(cf) \rangle$
- Given sequence $\langle a(abc)(ac)d(cf) \rangle$

Prefix	<u>Suffix</u> (Prefix-Based <u>Projection</u>)
$\langle a \rangle$	$\langle (abc)(ac)d(cf) \rangle$
$\langle aa \rangle$	$\langle (_bc)(ac)d(cf) \rangle$
$\langle ab \rangle$	$\langle (_c)(ac)d(cf) \rangle$

The Algorithm of PrefixSpan

- ❑ Input: A sequence database S , and the minimum support threshold min_sup
 - ❑ Output: The complete set of sequential patterns
 - ❑ Method: Call $\text{PrefixSpan}(\langle \rangle, 0, S)$
 - ❑ Subroutine $\text{PrefixSpan}(\alpha, l, S|\alpha)$
 - ❑ Parameters:
 - α : sequential pattern,
 - l : the length of α ;
 - $S|\alpha$: the α -projected database, if $\alpha \neq \langle \rangle$; otherwise; the sequence database S
-

The Algorithm of PrefixSpan

Method:

1. Scan $S|\alpha$ once, find the set of frequent items b such that:
 - a) b can be assembled to the last element of α to form a sequential pattern; or
 - b) $\langle b \rangle$ can be appended to α to form a sequential pattern.
 2. For each frequent item b , append it to α to form a sequential pattern α' , and output α' ;
 3. For each α' , construct α' -projected database $S|\alpha'$, and call PrefixSpan(α' , $l+1$, $S|\alpha'$).
-

Pseudocode of PrefixSpan (simple version)

PrefixSpan(a database D , a sequence S (initially empty $\langle \rangle$), $minsup$)

1. Scan D to find the support of each sequence starting with S that has one more item.
2. For each sequence R such that $sup(R) \geq minsup$
3. Output R
4. Create the projected database D_R of R by doing a projection with D
5. Call **PrefixSpan**($D_R, R, minsup$)

Mining Sequential Patterns by Prefix Projections

- Step 1: find length-1 sequential patterns
 - $\langle a \rangle, \langle b \rangle, \langle c \rangle, \langle d \rangle, \langle e \rangle, \langle f \rangle$
- Step 2: divide search space. The complete set of seq. pat. can be partitioned into 6 subsets:
 - The ones having prefix $\langle a \rangle$;
 - The ones having prefix $\langle b \rangle$;
 - ...
 - The ones having prefix $\langle f \rangle$

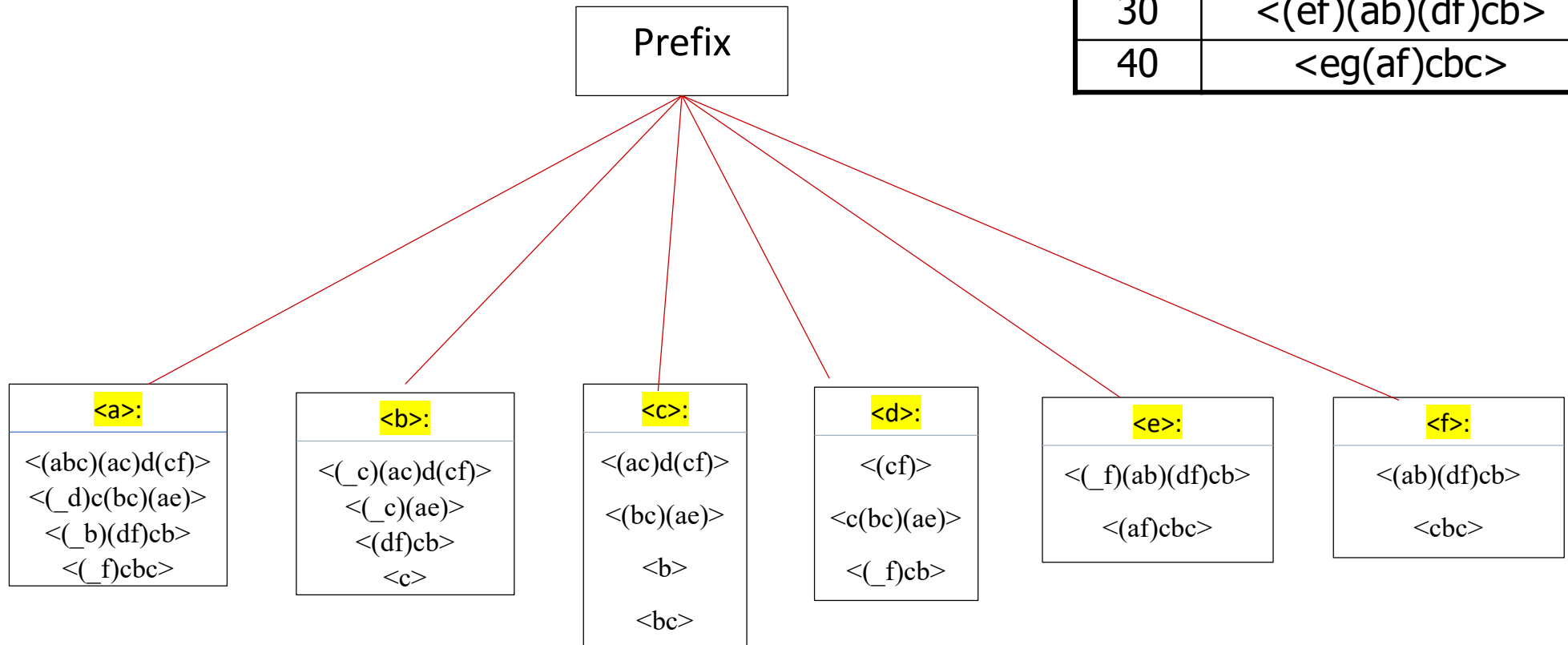
SID	sequence
10	$\langle a(abc)(ac)d(cf) \rangle$
20	$\langle (ad)c(bc)(ae) \rangle$
30	$\langle (ef)(ab)(df)cb \rangle$
40	$\langle eg(af)cbc \rangle$

Given support threshold $min_sup = 2$

Mining Sequential Patterns by Prefix Projections

<a>		<c>	<d>	<e>	<f>	<g>
4	4	4	3	3	3	1

SID	sequence
10	<a(abc)(ac)d(cf)>
20	<(ad)c(bc)(ae)>
30	<(ef)(ab)(df)cb>
40	<eg(af)cbc>



Finding Seq. Patterns with Prefix <a>

- Only need to consider projections w.r.t. <d>, find subsets of sequential patterns

d
<(cf)> <c(bc)(ae)> <(_f)cb>

<a>		<c>	<d>	<e>	<f>	<(_f)>
1	2	3	0	1	1	1

	<c>
2	1

<db>
<(_c)>

<dc>
<(bc)>

<dcb>
<>

Finding Seq. Patterns with Prefix <a>

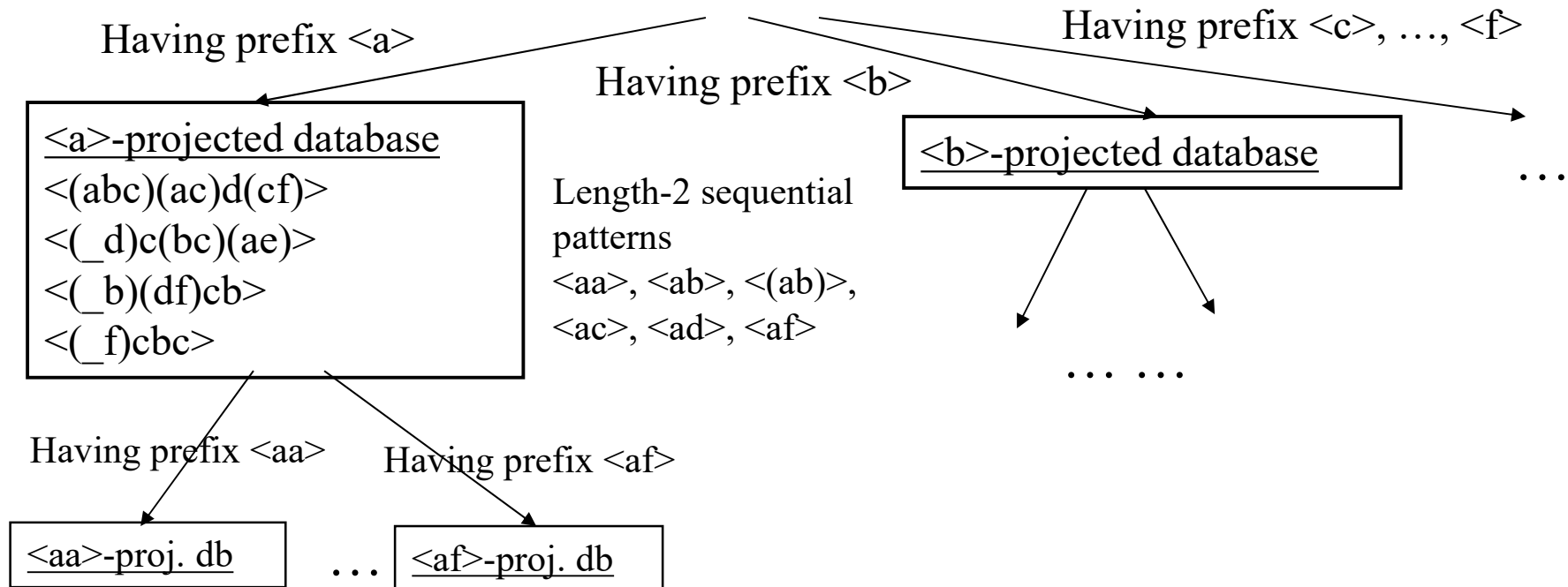
- Only need to consider projections w.r.t. <a>, find subsets of sequential patterns
 - <a>-projected database: <(abc)(ac)d(cf)>, <(_d)c(bc)(ae)>, <(_b)(df)cb>, <(_f)cbc>
- Find all the length-2 seq. pat. Having prefix <a>: <aa>, <ab>, <(ab)>, <ac>, <ad>, <af>
 - Further partition into 6 subsets
 - Having prefix <aa>;
 - ...
 - Having prefix <af>

Completeness of PrefixSpan

SDB

SID	sequence
10	<a(abc)(ac)d(cf)>
20	<(ad)c(bc)(ae)>
30	<(ef)(ab)(df)cb>
40	<eg(af)cbc>

Length-1 sequential patterns
<a>, , <c>, <d>, <e>, <f>



Efficiency of PrefixSpan

- ❑ No candidate sequence needs to be generated
- ❑ Projected databases keep shrinking
- ❑ Major cost of PrefixSpan: constructing projected databases
 - Can be improved by pseudo-projections

Ref: Mining Sequential Patterns

- [1] J. Han, M. Kamber, “Data Mining: Concepts and Techniques”, Second Edition, Morgan Kaufmann Publishers.
 - [2] P.N.Tan, M. Steinbach, V. Kumar, Introduction to Data Mining.
 - [3] R. Srikant and R. Agrawal. Mining sequential patterns: Generalizations and performance improvements. EDBT’96.
 - [4] M. Zaki. SPADE: An Efficient Algorithm for Mining Frequent Sequences. Machine Learning, 2001.
 - [5] J. Pei, J. Han, H. Pinto, Q. Chen, U. Dayal, and M.-C. Hsu. PrefixSpan: Mining Sequential Patterns Efficiently by Prefix-Projected Pattern Growth. ICDE’01 (TKDE’04).
-

